Aplicație: semafoare distribuite (3)

```
enum fel(V, P, ack);
chan opsem[1:n](int transm, fel op, int timp);
chan start[1:n](int timp);

process Utiliz[i= 1 to n] {
  int cl = 0;                    /* ceas logic */
  int ts;
  ...
  cl = cl + 1;
  broadcast opsem(i, V, cl);          /* operația V */
  ...
  cl = cl + 1;
  broadcast opsem(i, P, cl);          /* operația P */
  receive start[i](ts);
  cl = max(cl, ts)+1;
  ...
}
```

Aplicație: semafoare distribuite (4)

```
process Ajutor[i = 1 to n]{
    typedef struct{int transm, fel k, int ts) qelem;
    qelem qm[lmax]; /*coada ordonata dupa timestamp tm*/
    int cl = 0;
    int sem = valoare_inițială;
    int transm; fel k; int ts;
    while (true) {
        receive opsem[i](transm, k, ts);
        cl = max(cl, ts) + 1;
        if (k == P or k == V) {
            inserează (transm, k, ts) în locul corespunzător în qm;
            cl = cl + 1; broadcast opsem(i, ack, cl); }
        else if (k == ack) {
            înregistrează transmiterea unui ack;
            for [mesajele V complet confirmate] {
                scoate mesaj din qm;
                sem = sem + 1; }
            for [mesajele P complet confirmate st sem > 0] {
                scoate mesaj (transm, k, ts) din qm; // atentie la coliziuni
                sem = sem - 1;
                if (tansm == i) {cl = cl + 1; send start[i](cl);} }
        }
    }
}
```